НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ФІЗИКО-ТЕХНІЧНИЙ ІНСТИТУТ

КРИПТОГРАФІЯ

КОМП’ЮТЕРНИЙ ПРАКТИКУМ No4

Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

варіант 2

Виконали
Студенти групи ФБ-42:
Малишко Дарія
Сизова Марія

Мета та основні завдання роботи:

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Ми реалізували криптосистему RSA з нуля, включаючи генерацію криптостійких ключів, створення електронного цифрового підпису (ЕЦП) та захищений протокол розсилання ключів між двома абонентами. Усі математичні операції побудовані на власних функціях без використання сторонніх криптографічних бібліотек.

Завдання 1. Допоміжна арифметика

gcd(a, b): Класичний алгоритм Евкліда для знаходження найбільшого спільного дільника.

egcd(a, b): Розширений алгоритм Евкліда. Визначає коефіцієнти рівняння Безу, що необхідно для пошуку оберненого елемента.

mod_inverse(a, m): Обчислює обернений елемент за модулем. Це основа для генерації секретного ключа d.

mod_pow(x, a, m): Реалізація схеми Горнера для швидкого піднесення до степеня за модулем. Алгоритм читає показник у двійковому форматі, послідовно підносячи проміжні значення до квадрата та множачи, що дозволяє миттєво працювати з 256-бітними числами.

Завдання 2. Тести на простоту та генерація чисел

strong_pseudoprime(...): Виконує один раунд імовірнісного тесту Міллера-Рабіна, перевіряючи, чи є число сильно псевдопростим.

is_probable_prime(...): Комплексна перевірка. Спочатку число проходить «тест пробних ділень» на малі прості числа (для швидкого відсіву), а потім – k раундів тесту Міллера-Рабіна для забезпечення високої точності.

gen_prime(...): Базовий генератор, який шукає випадкове число заданої довжини, поки воно не пройде тести на простоту.

gen_safe_prime(...): Генерує «безпечне» просте число (число Софі Жермен) за формулою $p = 2p' + 1$, де p' також просте. Це захищає систему від специфічних криптоаналітичних атак на модуль RSA.

Завдання 3. Побудова криптосистеми RSA

GenerateKeyPair(p, q): Приймає згенеровані прості числа і створює ключі. Обчислює модуль $n = p \cdot q$ та функцію Ейлера $\phi(n) = (p - 1)(q - 1)$.

Відкрита експонента e за замовчуванням береться 65537 (якщо вона не взаємно проста з $\phi(n)$, підбирається інша). Секретний ключ d обчислюється через функцію mod_inverse.

Завдання 4. Шифрування, підпис та протокол розсилання

Encrypt та Decrypt: Виконують шифрування відкритого тексту та розшифрування криптотексту математичним піднесенням до степеня за модулем.

Sign та Verify: Створюють цифровий підпис (повідомлення шифрується секретним ключем відправника) та перевіряють його (підпис розшифровується відкритим ключем відправника).

SendKey(...): Відправник генерує секрет k, підписує його своїм секретним ключем d_A, а потім шифрує сам секрет і підпис відкритим ключем одержувача e_B.

ReceiveKey(...): Одержувач розшифровує дані своїм секретним ключем d_B і перевіряє автентичність підпису за допомогою відкритого ключа відправника e_A.

Код:

lab4.py > ...

```python
import random
import sys

SMALL_PRIMES = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47,
                53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

# Завдання 1. Допоміжна арифметика

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def egcd(a, b):
    x0, y0, x1, y1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

def mod_inverse(a, m):
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("Оберненого елемента не існує")
    return x % m

def mod_pow(x, a, m):
    y = 1
    for bit in bin(a)[2:]:
        y = y * y % m
        if bit == '1':
            y = y * x % m
    return y

```

```python
# Завдання 2. Тести на простоту та генерація чисел

def decompose(p):
    d, s = p - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    return d, s

def strong_pseudoprime(p, a, trace=None):
    d, s = decompose(p)
    x = mod_pow(a, d, p)
    if trace is not None:
        trace.append((0, x))
    if x == 1 or x == p - 1:
        return True
    for r in range(1, s):
        x = x * x % p
        if trace is not None:
            trace.append((r, x))
        if x == p - 1:
            return True
        if x == 1:
            return False
    return False

def is_probable_prime(p, k=20):
    if p < 2:
        return False, "p < 2"
    for q in SMALL_PRIMES:
        if p == q:
            return True, "Мале просте"
        if p % q == 0:
            return False, f"Ділиться на {q}"
    for _ in range(k):
        a = random.randint(2, p - 2)
        if gcd(a, p) != 1:
            return False, f"gcd(a, p) != 1"
        if not strong_pseudoprime(p, a):
            return False, "Не пройшло тест Міллера-Рабіна"
    return True, "Пройшло тести"

def gen_prime_interval(n0, n1, k=20, log=None):
    while True:
        x = random.randint(n0, n1)
        m = x if x % 2 == 1 else x + 1
        while m <= n1:
            ok, reason = is_probable_prime(m, k)
            if ok:
                return m
            if log is not None:
                log.append((m, reason))
            m += 2

def gen_prime(bits, k=20, log=None):
    return gen_prime_interval(1 << (bits - 1), (1 << bits) - 1, k, log)

def gen_safe_prime(bits, k=20, log=None):
    while True:
        p_prime = gen_prime(bits - 1, k, None)
        p = 2 * p_prime + 1
        ok, reason = is_probable_prime(p, k)
        if ok:
            return p
        if log is not None:
            log.append((p, f"Відхилено: {reason}"))

```

```
                log.append((p, f"Відхилено: {reason}"))

def miller_rabin_report(p, a):
    d, s = decompose(p)
    trace = []
    res = strong_pseudoprime(p, a, trace)
    print(f"p = {p}\np-1 = d * 2^s (s={s}, d={d})\nОснова a = {a}")
    for r, x in trace:
        print(f"x_{r} = {x}")
    print("Висновок: p псевдопросте" if res else "Висновок: p складене")
```

```
# Завдання 3. Побудова криптосистеми RSA

def GenerateKeyPair(p, q, e=65537):
    if p == q:
        raise ValueError("p і q однакові")
    n = p * q
    phi = (p - 1) * (q - 1)
    if not (1 < e < phi and gcd(e, phi) == 1):
        while True:
            e = random.randrange(3, phi - 1, 2)
            if gcd(e, phi) == 1:
                break
    d = mod_inverse(e, phi)
    return (n, e), (d, p, q)

```

lab4.py > ...

```python
# Завдання 4. Шифрування, підпис та протокол розсилання

def Encrypt(M, e, n):
    return mod_pow(M, e, n)

def Decrypt(C, d, n):
    return mod_pow(C, d, n)

def Sign(M, d, n):
    return mod_pow(M, d, n)

def Verify(M, S, e, n):
    return mod_pow(S, e, n) == M

def SendKey(k, d_A, n_A, e_B, n_B):
    S = Sign(k, d_A, n_A)
    k1 = Encrypt(k, e_B, n_B)
    S1 = Encrypt(S, e_B, n_B)
    return k1, S1

def ReceiveKey(k1, S1, d_B, n_B, e_A, n_A):
    k = Decrypt(k1, d_B, n_B)
    S = Decrypt(S1, d_B, n_B)
    if not Verify(k, S, e_A, n_A):
        raise ValueError("Підпис відправника не підтверджено")
    return k

def hx(x):
    return hex(x)


# Демонстрація виконання

def main():
    seed = int(sys.argv[1]) if len(sys.argv) > 1 else None
    random.seed(seed)
    BITS, K = 256, 20

    print("\n Завдання 2: Генерація безпечних простих чисел")
    primes, logs = {}, {}
    for name in ("p", "q", "p1", "q1"):
        logs[name] = []
        primes[name] = gen_safe_prime(BITS, K, logs[name])
    p, q, p1, q1 = primes["p"], primes["q"], primes["p1"], primes["q1"]

    if p * q > p1 * q1:
        p, q, p1, q1 = p1, q1, p, q

    print(f"p = {p}\nq = {q}\np1 = {p1}\nq1 = {q1}")

    print("\n Завдання 3: Генерація ключів RSA")
    pubA, privA = GenerateKeyPair(p, q)
    pubB, privB = GenerateKeyPair(p1, q1)
    nA, eA = pubA; dA = privA[0]
    nB, eB = pubB; dB = privB[0]

    print(f"Абонент A:\n  n = {hx(nA)}\n  e = {eA}\n  d = {hx(dA)}")
    print(f"Абонент B:\n  n = {hx(nB)}\n  e = {eB}\n  d = {hx(dB)}")

    print("\n Завдання 4: Підпис та протокол розсилання ключів")
    M = random.randint(2, nA - 1)
    C = Encrypt(M, eA, nA)
    S = Sign(M, dA, nA)
    print(f"Повідомлення M: {hx(M)}\nЗашифроване C: {hx(C)}\nПідпис S: {hx(S)}")
    print(f"Перевірка підпису: {Verify(M, S, eA, nA)}")

    k = random.randint(1, nA - 1)
```

```
195
196         k = random.randint(1, nA - 1)
197         print(f"\nВідправка секрету k: {hx(k)}")
198         k1, S1 = SendKey(k, dA, nA, eB, nB)
199         k_rec = ReceiveKey(k1, S1, dB, nB, eA, nA)
200         print(f"Отриманий абонентом В секрет: {hx(k_rec)}")
201         print(f"k_A == k_B: {k == k_rec}")
202
203     if __name__ == "__main__":
204         main()
```

Вивід:

```
python -u "c:\Users\Admin\Desktop\crypto26-27\Lab1\FB-42_Syzova_Malyshko\lab4.py"

 Завдання 2: Генерація безпечних простих чисел
p = 75585374110596726666081175010922343669123968842988699582284439239115309064403
q = 69172587070711371301893221191395475225550886592462886531326639286073748148147
p1 = 10637660619965671503261665568567021776458451979732181566554949608076856139242 7
q1 = 69187546139115949022313071088742210206973127956934876709939908907421523042399

 Завдання 3: Генерація ключів RSA
Абонент A:
  n = 0x63d40febc0b84ad4a216c0a7fa7eacc80b470696629c8440b775749525cd047fe062a9047744a6e0bb00752e2a3134193260335d5832d3c79b21ecd279899089
  e = 65537
  d = 0x4806b859fbd9bc3c009af3f3fedb28e4c8a63d4d81e6b5d14fccd209ce954699ea2d1b0d3a597c707a64489c5556a207178b3df0ce008bc4039b3078816f02d5
Абонент B:
  n = 0x8c869e978869151a66c50521d5b6fe2960beccd5047781e4d3cded0bb059972cc9bacfdac1a77dc302c6c7a3162cba61266e61ccc5b08a7e300ed08a1cca4f5
  e = 65537
  d = 0x7f0b017bbb7e4ceed6744e247da2f77a8f154ff05a2092ceca433aa9783b22b2d0b08b3927c73d2b378e7342fee218e20fe604e35bc8aa9abf8199ff7d192cad

 Завдання 4: Підпис та протокол розсилання ключів
Повідомлення M: 0x4d87530b4a8f21d0f4d70757f0a046d7b308bf6647bc53aac4178a2776b63fbaf01817e6b654b7f65b9244e267d635163b1057e8950318530b89b86b8cc73e4c
Зашифроване C: 0x5be905d46b9d85d68e6eea37573d641679aa070fb56b779c4d38d21eb05ce64e24db9a93b0f6d1694e65a554aeac78f1fb5e78b85717ac5da2f18729823401b6
Підпис S: 0x3fafe8992cb4259055cf8aa090f854d9a44ed12245cdb9b61e69071e35d0a19b5cc3b09b039e6d49270e6dbfee644b6f4d2bafdb3dadf0f1628adef5d08ad30d
Перевірка підпису: True

Відправка секрету k: 0x20f357fd18b4d2fa7ec4e0fee586fae6f3f305edb3f108c4e3f602b65cad853495b5cf6de94ed94249a712c35dfd6bb8153cf1fafd09fb1421cc44f40d33c3cd
Отриманий абонентом В секрет: 0x20f357fd18b4d2fa7ec4e0fee586fae6f3f305edb3f108c4e3f602b65cad853495b5cf6de94ed94249a712c35dfd6bb8153cf1fafd09fb1421cc44f40d33c3cd
k_A == k_B: True
PS C:\Users\Admin\Desktop\crypto26-27\Lab1\FB-42_Syzova_Malyshko>
```

Висновок:

Результати виконання скрипта в терміналі підтверджують абсолютну коректність математичної моделі. Згенеровані великі числа p, q, p_1, q_1 пройшли всі перевірки на безпеку, а створені ключові пари мають правильний криптографічний розмір.

Блок цифрового підпису відпрацював безпомилково: функція верифікації повернула True, підтвердивши цілісність повідомлення. Протокол розсилання ключів також завершився повним успіхом – секрет k, надісланий абонентом А, ідеально збігається із секретом, розшифрованим абонентом Б (k_A == k_B: True). Це доводить, що розроблена система гарантує як надійну конфіденційність передачі даних, так і безперечну автентичність відправника.